Technical Decisions

Building the MVP

Frontend: Next.js

In a previous full-stack project I did for CS316, the focus was more on how the backend and database connected to each other and how the backend retrieved and stored data in the database. There was less focus on the frontend, so I really haven't had much experience developing an interactive UI. I wanted to learn more about frontend development, and I had read and heard that React and React frameworks can make building interactive user interfaces more efficient by allowing the UI to be broken into reusable components. I am choosing to use Next.js overreact on its own, because I think having the ability to explicitly control server-side rendering and client-side rendering is a plus, but ultimately, I don't expect these to be major factors in the early stages of the project.

Backend: FastAPI

I wanted to use a Python backend because I am familiar with Python, and using the language would build on what I already learned from my previous full-stack project. I am most comfortable with Python, and using a separate Python backend also lets me keep the backend and frontend as separate parts of the application rather than learning Next.js's backend/API functionality at the same time. The two tools I was deciding between were Flask and FastAPI.

Earlier, I planned on using FastAPI because I read that it had built-in utilities for things like request validation and implementing authentication and authorization. The fact that FastAPI is specifically designed around building APIs also caught my attention, especially because the backend will primarily communicate with the Next.js frontend through API requests. There were also many tutorials and examples showing how to connect a React/Next.js frontend to a FastAPI backend, so I knew there would be resources available when it came to integrating the two.

In hindsight, it might have been a good idea to stick with Flask, since I used Flask in a previous project and am already familiar with it. Flask is also a more mature framework, so there is an argument for using something I already know rather than introducing another framework. I'm not sure if I'll want to switch back to Flask later down the line, but right now I am planning to use FastAPI.

Database Management System: PostgreSQL

I chose PostgreSQL because that is what I used for my previous full-stack application, so I am already comfortable working with it. Using PostgreSQL also allows me to build on what I learned from that project rather than having to learn a new database system.

Testing

Backend Testing

Because I'm using Python as my backend, the two testing frameworks I considered were Pytest and Unittest. Both are widely used, so there are many resources for learning how to use either. Although Unittest is Python's built-in testing framework, other developers have noted that Pytest has simpler and more readable syntax, which may make it easier for beginner developers to learn. Since I am learning a lot throughout this Independent Study, I would prefer a framework that I can likely have an easier time comprehending. Pytest also supports unit testing and integration testing, both of which I want to use for my application.

Frontend Testing

Since my application uses Next.js for the frontend, I am considering Jest or Vitest as the test runner, along with React Testing Library (RTL) for unit and component testing. These tools would allow me to test TypeScript methods and individual components.

I had a hard time deciding whether to use Jest or Vitest because my priorities are not based on performance-related metrics, such as speed, or the number of additional features a framework offers. Instead, I prioritize whichever framework is the most straightforward to set up and requires the least manual configuration from me as the developer. I want to focus my time on identifying which areas of the application would benefit from automated tests and writing those tests.

Although Jest is a mature testing framework with extensive documentation, some developers have reported difficulties configuring Jest to run TypeScript tests and resolve path aliases. Although recent versions of Next.js configure parts of Jest for you through `next/jest`, I had difficulty finding documentation explicitly stating that next/jest automatically configures Jest to run TypeScript tests. This makes me hesitant to use Jest because I don't want to potentially have to learn and implement additional configurations before I can start writing tests.

Vitest offers TypeScript support out of the box without requiring additional configuration. I am leaning toward using Vitest and RTL for frontend testing primarily for this reason. Vitest is also significantly faster than Jest and works well with Vite-based applications, but these factors are not particularly important to me in this case.

If I could find documentation confirming that `next/jest` automatically configures Jest to run TypeScript tests, I would lean toward using Jest because next/jest handles other configurations automatically. While I don't anticipate benefiting significantly from these additional

configurations initially, it would be nice to have them automatically handled for compatibility with Next.js, if the types of tests I write later require them later on.

End-To-End Testing

Both Cypress and Playwright are popular, modern end-to-end (E2E) web testing frameworks. Due to the availability of resources for learning either framework, I was initially deciding between these two. Ultimately, I am leaning toward Playwright because it supports multiple programming languages, including Python and TypeScript, giving me the flexibility to write tests in Python if I decide to do so. Playwright also offers a wide range of built-in testing features for free, making it a viable option for testing my application.